VERIQTA

VERIQTA

100 DevOps Commands

The Ultimate Reference


Linux


Git


Docker


Kubernetes



AWS


Terraform


CI/CD


System Administration


Networking


Security


Cloud


Logs


Troubleshooting


Monitoring


Automation


@veriqta


@veriqta


@veriqta


@veriqta


@veriqta


@veriqta

Essential Commands for Real DevOps Work

Linux Navigation and Working Context

VERIQTA

Learn to identify your location, explore directories, and understand the environment before making changes.

1 pwd

- Show the current working directory.
- Tells you exactly where you are in the filesystem.
- Useful before running commands or editing files.

Example

```
$ pwd
/home/ubuntu/projects
```

When to Use

- Before running scripts
- Before creating or editing files
- To confirm your current location

2 ls -lah

- List files and directories in detail.
- Shows permissions, sizes, timestamps and hidden files.
- Helps you understand what is in the current directory.

Example

```
$ ls -lah
total 32K
drwxr-xr-x  3 ubuntu ubuntu 4.0K Oct 14 10:15 .
drwxr-xr-x  5 ubuntu ubuntu 4.0K Oct 14 09:50 ..
-rw-r--r--  1 ubuntu ubuntu  512 Oct 14 10:12 app.py
drwxr-xr-x  2 ubuntu ubuntu 4.0K Oct 14 10:11 configs
```

When to Use

- Check file permissions
- View hidden files (starting with .)
- Inspect file sizes and dates
- Verify files after a download or deployment

3 cd

- Change the current directory.
- Move into another folder in the filesystem.
- Use .. to move up one level.

Example

```
$ cd /var/log
$ pwd
/var/log
$ cd ..
$ pwd
/var
```

When to Use

- Navigate between directories
- Move to a project or config folder
- Use **cd ..** to go up one level
- Use **cd ~** to go to your home directory
- Use **cd -** to return to the previous directory

4 realpath

- Resolve a file or directory to its absolute path.
- Removes . and .. from the path.
- Useful when you are working with symbolic links or relative paths.

Example

```
$ realpath ./app/../configs
/home/ubuntu/configs
```

When to Use

- Get the full absolute path
- Verify the actual location of a file
- Useful in scripts and automation
- Helps avoid path-related errors

5 find

- Locate files using names, types or other criteria.
- Search recursively in directories.
- Very useful when you do not know the exact location of a file.

Example

```
$ find /var/log -name "*.log"
/var/log/syslog
/var/log/auth.log
/var/log/app/app.log

$ find . -type d -name "configs"
./app/configs
```

When to Use

- Search for files by name
- Find files by type (file or directory)
- Locate configuration files
- Useful on large systems
- Helpful when troubleshooting missing files

Engineering Focus

Avoid running commands against the wrong directory or environment. Always confirm your current location before making changes, especially on servers, production systems, or shared environments.

Common Beginner Mistakes

- Running commands in the wrong directory.
- Editing or deleting the wrong files.
- Forgetting hidden files exist.
- Using relative paths without understanding them.
- Assuming you are in your home directory.

Try This

1. Run **pwd** and note the output.
2. Use **ls -lah** to inspect the current directory.
3. Create a new directory and move into it.
4. Use **realpath** to get the absolute path.
5. Use **find** to locate a file by name.

Files, Directories, and Safe File Operations

VERIQTA

Learn to create, inspect, copy, move, and remove files during configuration and deployment work.

1 mkdir -p

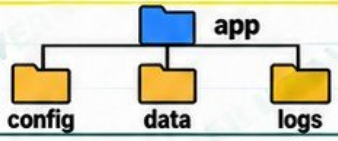
- Create nested directories in one command.
- Creates parent directories if they do not exist.
- Useful for setting up project and deployment folder structures.

Example

```
$ mkdir -p /app/data/logs
$ tree /app
/app
└── data
    └── logs
```

When to Use

- Set up application directories
- Create nested folders for config, logs, or backups
- Automate environment setup



2 touch

- Create an empty file.
- Update the timestamp of an existing file.
- Useful for creating configuration files or testing file paths.

Example

```
$ touch app.conf
$ ls -l app.conf
-rw-r--r-- 1 ubuntu ubuntu
0 Oct 14 10:20 app.conf
```

When to Use

- Create new configuration files
- Update file timestamp
- Prepare files for scripts and deployments



3 cp -a

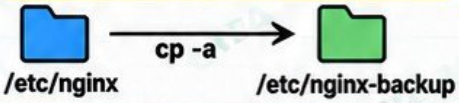
- Copy files or directories.
- Preserves important attributes such as permissions, ownership, and timestamps.
- Use -a for safe, complete copies.

Example

```
$ cp -a /etc/nginx
/etc/nginx-backup
$ ls -ld /etc/nginx-backup
drwxr-xr-x 7 root root
4096 Oct 14 10:25 /etc/nginx-backup
```

When to Use

- Back up configuration directories
- Copy application files
- Preserve permissions and timestamps
- Migrate files between environments



4 mv

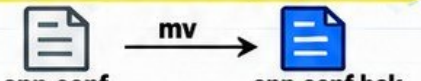
- Move or rename files and directories.
- Useful for organizing files.
- Commonly used during deployments to rename old and new versions.

Example

```
$ mv app.conf app.conf.bak
$ mv /tmp/app /opt/app
$ ls -l app.conf.bak
-rw-r--r-- 1 ubuntu ubuntu
1234 Oct 14 10:30 app.conf.bak
```

When to Use

- Rename files
- Move files to a new directory
- Organize project structures
- Replace old files during deployment



5 rm -i

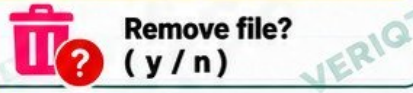
- Remove files with an interactive confirmation.
- Prompts you before deleting each file.
- Helps prevent accidental deletion.

Example

```
$ rm -i old.log
rm: remove regular file 'old.log'?
y
$ ls old.log
ls: cannnoot access 'old.log':
No such file or directory
```

When to Use

- Safely remove files
- Avoid accidental deletion
- Useful in interactive sessions
- Use with caution in scripts



Engineering Focus

Understand overwrites, deletion risks, and why backups matter. Always confirm file paths and keep important files backed up before making changes.

Common Beginner Mistakes

- Using the wrong path
- Overwriting important files
- Deleting files without confirmation
- Confusing relative and absolute paths
- Copying files without preserving permissions
- Removing directories recursively by mistake

Try This

1. Create a nested directory for a project.
2. Create a new file and check its timestamp.
3. Copy a configuration directory as a backup.
4. Rename the file you created.
5. Remove a test file using `rm -i`.

Reading Logs and Investigating Text

VERIQTA

Learn how engineers examine application logs, configuration files, and diagnostic output.

1 cat

- Display the full contents of a file.
- Useful for small files.
- Commonly used to quickly view configuration files and logs.

Example

```
$ cat /etc/hosts
127.0.0.1    localhost
127.0.1.1    ubuntu

# The following lines are examples
192.168.1.10   app-server
```

When to Use

- View small log or config files
- Check system configuration
- Inspect file content quickly
- Verify changes after editing

/etc/hosts

2 less

- Navigate large files page by page.
- Does not load the entire file into memory.
- Lets you search, move forward, backward, and exit easily.

Example

```
$ less /var/log/syslog
... use these keys ...
SPACE   → next page
b       → previous page
/error  → search for "error"
q       → quit
```

When to Use

- View large log files
- Read configuration files
- Search inside a file
- Avoid scrolling through thousands of lines

/var/log/syslog

4 head

- Show the first few lines of a file.
- Useful for quickly checking a file's content.
- Commonly used to inspect log files or configuration files.

Example

```
$ head -n 5 /var/log/nginx/access.log
192.168.1.10 - - [14/Oct/2026:10:12:01]
192.168.1.12 - - [14/Oct/2026:10:12:05]
192.168.1.15 - - [14/Oct/2026:10:12:08]
192.168.1.20 - - [14/Oct/2026:10:12:11]
192.168.1.25 - - [14/Oct/2026:10:12:14]
```

When to Use

- Quickly check the start of a file
- Verify file format or headers
- Inspect log entries
- Confirm a file is being written

/var/log/nginx/access.log

4 tail -f

- Follow appended output in real time.
- Commonly used for application logs.
- Very useful for troubleshooting live issues.

Example

```
$ tail -f /var/log/nginx/error.log
2026/10/14 10:15:01 [error] 1234#
2026/10/14 10:15:05 [warn]  1250#
2026/10/14 10:15:07 [error] 1260#
2026/10/14 10:15:12 [info]  1270#
2026/10/14 10:15:18 [error] 1280#
...
```

When to Use

- Monitor live application logs
- Watch new log entries as they happen
- Troubleshoot errors in real time
- Verify if a service is writing logs

/var/log/nginx/error.log

5 grep -n

- Search for specific text in files.
- Shows matching line numbers with -n.
- Useful for finding errors, keywords, or specific patterns.

Example

```
$ grep -n "error" /var/log/syslog
15:Oct 14 10:05:12 server1 app[123]: error
42:Oct 14 10:10:33 server1 db[456]: error
87:Oct 14 10:15:01 server1 nginx[789]: error
120:Oct 14 10:18:25 server1 app[321]: error
```

When to Use

- Find errors in log files
- Search configuration files
- Locate specific events
- Quickly identify relevant lines

/var/log/syslog

Engineering Focus

Find evidence of failures without changing the system. Always read logs and files carefully before making any changes.

Common Beginner Mistakes

- Using cat on very large files
- Forgetting to exit less (press q)
- Not using -n with grep
- Ignoring error messages in logs
- Misreading timestamps or log context

Try This

1. Use **cat** /etc/os-release
2. Use **less** /var/log/syslog
3. Use **head -n 10** /var/log/syslog
4. Use **tail -f** /var/log/syslog (press Ctrl+C to stop)
5. Use **grep -n "error"** /var/log/syslog

Linux Users, Permissions, and Privilege

VERIQTA

Understand who can access files, run services, and perform administrative actions.

1 whoami

- Identify your current user.
- Shows the username you are logged in as.
- Helpful to confirm which account you are using before making changes.

Example

```
$ whoami
ubuntu
```

When to Use

- Confirm your current user after logging in.
- Before running commands that require specific users.
- Verify you are not running as root unless necessary.

2 id

- Inspect user and group identities.
- Shows your user ID (UID), group ID (GID) and groups.
- Useful for troubleshooting permission issues.

Example

```
$ id
uid=1000(ubuntu)
gid=1000(ubuntu)
groups=1000(ubuntu),4(adm),
24(cdrom),27(sudo),100(users)
```

When to Use

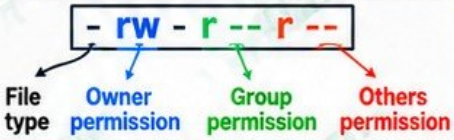
- Check which groups you belong to.
- Understand why you can or cannot access certain files or services.
- Verify if you are in the correct group (e.g., docker, sudo, www-data).

3 ls -l

- Examine file ownership and permissions.
- Shows file type, permissions, owner, group, size and date.
- Helps you understand who can read, write, or execute a file.

Example

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root  1984
Oct 12 10:25 /etc/passwd
```



When to Use

- Check file permissions.
- Identify the owner and group of a file or directory.
- Troubleshoot permission denied errors.
- Verify permissions before editing configuration files.

4 chmod

- Modify file permissions.
- Changes who can read (r), write (w), or execute (x).
- Use symbolic (u, g, o) or numeric (e.g. 755) modes.

Example

```
# Symbolic mode
$ chmod u+x script.sh

# Numeric mode
$ chmod 755 script.sh

$ ls -l script.sh
-rwxr-xr-x 1 ubuntu ubuntu 1024
Oct 12 10:25 script.sh
```

When to Use

- Make a script executable.
- Restrict access to sensitive files.
- Set correct permissions for configuration files.
- Ensure least privilege access.

Avoid chmod 777
It gives full access to everyone and is a security risk.

5 sudo -l

- Inspect permitted sudo operations.
- Shows which commands you can run with sudo.
- Helps you understand your privileges without guessing.

Example

```
$ sudo -l
Matching Defaults entries for ubuntu
    env_reset, mail_badpass,
    secure_path=/usr/sbin:/usr/bin
User ubuntu may run the following
commands on this host:
    (ALL) /bin/systemctl restart nginx
    (ALL) /usr/bin/docker *
```

When to Use

- Check which commands you are allowed to run with sudo.
- Troubleshoot "permission denied" errors.
- Understand your access level on a server.
- Verify if you have the required privileges for an operation.

Engineering Focus

Diagnose permission-denied errors without resorting to chmod 777. Always understand ownership, groups, and required privileges before making changes.

Common Beginner Mistakes

- Using chmod 777 to "fix" permission errors.
- Editing files without understanding ownership.
- Running commands as root unnecessarily.
- Not being in the correct group (e.g. docker).
- Assuming sudo gives unlimited access.

Try This

1. Run `whoami` and `id`.
2. Use `ls -l` on a file or directory.
3. Change a script to be executable using `chmod`.
4. Run `sudo -l` and review the allowed commands.
5. Verify the permission change using `ls -l` again.

Processes, CPU, and Memory Investigation

Learn to identify resource-intensive processes and investigate system slowdowns.

VERIQTA

1 ps aux

- Inspect running processes.
- Shows detailed information about all processes.
- Helps you find CPU or memory hungry processes.
- Useful for troubleshooting application issues.

Example

```
$ ps aux | head -5
USER     PID %CPU %MEM    VSZ   RSS TTY
root       1  0.0  0.1  16680  1300 ?
ubuntu   841  0.0  0.2  40284  8124 ?
ubuntu  1022  1.5  2.1 517432 87624 ?
ubuntu  2043  0.0  0.3  28820 11208 ?
```

When to Use

- Find processes using high CPU or memory.
- Investigate which application is running.
- Check processes started by specific users.
- Useful before terminating a process.

2 top

- Monitor live CPU and memory activity.
- Shows real-time system performance.
- Lets you sort processes, view usage and identify resource bottlenecks.

Example

```
$ top
top - 10:15:22 up 2 days,  3:14,  1 user
Tasks: 215 total,   1 running, 214 sleeping
%Cpu(s): 12.5 us,  3.1 sy,  0.0 ni, 84.4 id
MiB Mem :   3932.0 total,    512.1 free,
MiB Swap:   2048.0 total,   2048.0 free,

  PID  USER      %CPU  %MEM  COMMAND
 1022  ubuntu    25.3   2.1  node
 2043  ubuntu    10.1   1.3  python
 3112  root       1.5   0.5  docker
```

When to Use

- Monitor live system activity.
- Identify processes consuming high CPU or memory.
- Troubleshoot performance issues.
- Useful during an active incident.

Press q to exit top.

3 free -h

- Inspect memory availability.
- Shows total, used, free and cached memory in human readable format.
- Helps you understand if the system is under memory pressure.

Example

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       2.1Gi       512Mi       120Mi       1.2Gi       1.4Gi
Swap:          2.0Gi          0B       2.0Gi
```

When to Use

- Check if the system is running low on memory.
- Verify swap usage.
- Investigate out-of-memory (OOM) issues.
- Useful before running resource-intensive tasks.

4 uptime

- Check system uptime and load averages.
- Shows how long the system has been running.
- Displays load averages, which indicate CPU demand.
- Useful for quickly assessing system health.

Example

```
$ uptime
  10:15:22 up 2 days,  3:14,  1 user,
  load average: 0.35, 0.28, 0.21
```

What is Load Average?

- 1 minute: 0.35
- 5 minutes: 0.28
- 15 minutes: 0.21

} Lower is better.

- A value higher than the number of CPU cores may indicate high load.

When to Use

- Quickly check if the system is under heavy load.
- Understand how long the system has been running.
- Useful during performance investigations.
- Compare load averages over time to detect trends.

5 kill -TERM

- Request graceful process termination.
- Sends the TERM signal, allowing the process to shut down cleanly.
- Commonly used to stop applications.
- Use with a process ID (PID).

Example

```
$ kill -TERM 2043
$ ps aux | grep 2043
ubuntu  3250  0.0  0.0 7986  972 pts/0
grep    2043
```

When to Use

- Stop a process gracefully.
- Give the application time to clean up.
- Prefer before using a forceful kill.
- Useful dumb servers, workers and application services.

If the process does not stop, a stronger signal (e.g. -KILL) may be required.

Engineering Focus

Investigate resource pressure before restarting or terminating applications. Use these commands to understand what is happening on the system and make informed decisions.

Common Beginner Mistakes

- Killing the wrong process.
- Ignoring high memory or CPU usage.
- Not understanding load averages.
- Restarting services without investigating.
- Using kill -9 immediately instead of trying a graceful shutdown first.

Try This

1. Run ps aux and identify the top CPU process.
2. Open top and observe live system activity.
3. Run free -h and note memory usage.
4. Check uptime and interpret the load average.
5. Gracefully stop a test process using kill -TERM.

Services, System Logs, and Storage

VERIQTA

Learn how to determine whether a Linux host and its services are healthy.

1 systemctl status

- Inspect a systemd service.
- Shows whether the service is active, inactive, or failed.
- Displays key information such as process ID, memory usage, and recent logs.

Example

```
$ systemctl status nginx
● nginx.service - A high performance
  web server
   Loaded: loaded (/usr/lib/systemd/..)
   Active: active (running) since …
 Main PID: 1234 (nginx)
   Memory: 12.3M
   CGroup: /system.slice/nginx.service
           ├─1234 nginx: master process
           └─1235 nginx: worker process
```

When to Use

- Check if a service is running.
- Verify service status after configuration changes.
- Troubleshoot failed services.
- Confirm restart or reload was successful.

nginx.service

2 journalctl -u

- Read logs for a specific service.
- Shows detailed log messages from the systemd journal.
- Helps you investigate service failures and errors.
- Useful for real-time and historical log analysis.

Example

```
$ journalctl -u nginx -n 10
Oct 14 10:21:11 server1 nginx[1234]:
Oct 14 10:21:11 server1 nginx[1234]:
Oct 14 10:21:12 server1 nginx[1234]:
Oct 14 10:21:12 server1 systemd[1]:
Oct 14 10:21:13 server1 nginx[1235]:
```

When to Use

- View recent logs for a service.
- Investigate why a service failed to start.
- Monitor live logs with `-f`.
- Identify errors, warnings, and restart loops.

/var/log/journal/

3 df -h

- Inspect filesystem capacity.
- Shows total, used, and available spacs in a human readable format.
- Helps you quickly identify full or nearly full disks.
- Useful for preventing disk space issues.

Example

```
$ df -h
Filesystem  Size  Used Avail Use% Mounted on
/dev/sda1    50G   20G   28G  42% /
tmpfs       2.0G     0  2.0G   0% /dev/shm
/dev/sdb1   200G  180G   20G  90% /data
```

When to Use

- Check overall disk usage.
- Identify partitions that are running out of space.
- Verify if there is enough space for deployments.
- Use before investigating application errors.

/ , /data

4 du -sh

- Measure directory storage consumption.
- Shows the total size of a directory in a human readable format.
- Helps you find large directories and files.
- Useful for investigating disk space usage.

Example

```
$ du -sh /var/log
3.2G    /var/log
$ du -sh /var/* | sort -h
12M     /var/cache
320M    /var/lib
3.2G    /var/log
5.1G    /var/www
```

When to Use

- Find which directories are using the most space.
- Investigate unexpected disk growth.
- Identify large log files or application data.
- Use before cleaning up disk space.

/var/log

5 lsblk

- Inspect block devices and their relationships.
- Shows disks, partitions, and mount points.
- Helps you understand the system's storage layout.
- Useful for troubleshooting storage and mount issues.

Example

```
$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
sda      8:0    0  100G  0 disk
├─sda1   8:1    0   50G  0 part  /
├─sda2   8:2    0   50G  0 part  /data
sdb      8:16   0  200G  0 disk
└─sdb1   8:17   0  200G  0 part  /backup
```

When to Use

- View disk and partition information.
- Check which devices are mounted.
- Verify new disks after attachment.
- Understand storage layout before making changes.

/dev/sda , /dev/sdb

Engineering Focus

Investigate service failures and full disks using evidence. Always check service logs, disk usage, and storage layout before making changes.

Common Beginner Mistakes

- Assuming a service is running without checking.
- Ignoring error messages in logs.
- Not checking disk space before deployments.
- Deleting files without confirming their importance.
- Misinterpreting disk usage due to hidden files.

Try This

1. Run `systemctl status ssh`.
2. View the last 10 logs using `journalctl -u ssh -n 10`.
3. Check disk usage with `df -h`.
4. Find the largest directory in /var using `du -sh /var/*`.
5. Inspect your storage layout with `lsblk`.

IP Addresses, Routes, and Connectivity

VERIQTA

Understand how Linux hosts communicate across networks.

1 ip addr

- Inspect network interfaces and IP addresses.
- Shows interface names, IP addresses, MAC addresses and interface status.
- Useful for verifying network configuration and troubleshooting connectivity issues.

Example

```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP>
   inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP>
   inet 192.168.1.10/24 brd 192.168.1.255
   link/ether 00:15:5d:01:23:45
   inet6 fe80::215:5dff:fe01:2345/64
```

When to Use

- Check which network interfaces are available.
- Find the IP address of an interface.
- Verify if an interface is up or down.
- Troubleshoot missing or incorrect IP configuration.



Network interface (e.g. eth0, ens33, wlan0)

2 ip route

- Inspect routing decisions.
- Shows the default gateway and network routes.
- Helps you understand where network traffic is sent.
- Useful for troubleshooting connectivity and routing issues.

Example

```
$ ip route
default via 192.168.1.1 dev eth0
proto dhcp metric 100
192.168.1.0/24 dev eth0 proto kernel
scope link src 192.168.1.10
10.0.0.0/16 via 192.168.1.254 dev eth0
```

When to Use

- Check the default gateway.
- Verify network routes.
- Troubleshoot why traffic is not reaching a destination.
- Identify which interface is used for a specific network.



default via 192.168.1.1

3 ping

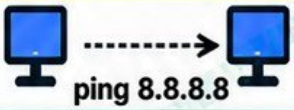
- Test ICMP reachability where permitted.
- Checks if a host is reachable on the network.
- Shows response time (latency).
- Useful for quick connectivity tests.

Example

```
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=12.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=117 time=11.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=117 time=12.1 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=117 time=12.3 ms
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 11.9/12.1/12.4/0.2 ms
```

When to Use

- Test basic network connectivity.
- Verify if a host or IP address is reachable.
- Check network latency.
- Useful for initial troubleshooting.



ping 8.8.8.8

4 tracepath

- Inspect a network path where supported.
- Shows the route packets take to reach a destination.
- Helps identify where connectivity issues occur.
- Useful when ping works but applications still fail.

Example

```
$ tracepath 8.8.8.8
 1:    192.168.1.1      1.234ms
 2:    10.0.0.1         12.451ms
 3:    172.16.0.1       18.672ms
 4:    8.8.8.8          24.315ms
Resume: pmtu 1500 hops 4  back 4
```

When to Use

- Find the path to a destination.
- Identify where delays or failures occur.
- Useful when ICMP is limited.
- Helps differentiate between local and external network issues.



tracepath 8.8.8.8

5 ss -tuln

- Inspect listening TCP and UDP sockets.
- Shows which ports are open.
- Helps you identify which services are running.
- Useful for troubleshooting connectivity and firewall issues.

Example

```
$ ss -tuln
Netid  State    Local Address:Port   Peer Address:Port
tcp    LISTEN     0.0.0.0:22          0.0.0.0:*
tcp    LISTEN   127.0.0.1:25          0.0.0.0:*
tcp    LISTEN     0.0.0.0:80          0.0.0.0:*
udp    UNCONN     0.0.0.0:68          0.0.0.0:*
udp    UNCONN   127.0.0.1:323         0.0.0.0:*
```

When to Use

- Check which ports are listening.
- Verify if a service is running and accessible.
- Troubleshoot connectivity issues.
- Confirm the correct interface and port.



Ports (e.g. 22, 80, 443)

Engineering Focus

Distinguish application failures from routing, interface, and port problems. Use these commands to identify where the issue is happening before making changes.

Common Beginner Mistakes

- Assuming ping always works (it can be blocked).
- Misreading routing information.
- Ignoring the correct network interface.
- Not checking if a service is listening on the right port.
- Confusing network issues with application issues.

Try This

1. Run ip addr and identify your IP address.
2. Check the default gateway using ip route.
3. Ping a public IP address (e.g. 8.8.8.8).
4. Use tracepath to a website.
5. Run ss -tuln and find which ports are open.

DNS, HTTP, and API Troubleshooting

VERIQTA

Learn how to investigate the path from a domain name to a responding web service.

1 dig

- Query DNS records.
- Shows detailed DNS information such as A, AAAA, CNAME, MX and more.
- Helps you investigate DNS resolution issues.
- Useful for troubleshooting domain and connectivity problems.

Example

```
$ dig veriqta.com
;; ANSWER SECTION:
veriqta.com.    300    IN    A    104.21.16.1
veriqta.com.    300    IN    A    172.67.172.10
;; Query time: 23 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Tue Oct 14 10:25:31 UTC 2026
;; MSG SIZE  rcvd: 84
```

When to Use

- Check what IP address a domain name resolves to.
- Verify DNS records (A, AAAA, CNAME, MX, etc.).
- Troubleshoot DNS-related connectivity issues.
- Confirm if DNS changes have propagated.

veriqta.com

2 getent hosts

- Inspect hostname resolution through system name services.
- Uses your system's configured name services (DNS, /etc/hosts. etc.).
- Shows the IP address resolved for a hostname.
- Useful for checking what your system actually sees.

Example

```
$ getent hosts veriqta.com
104.21.16.1      veriqta.com
172.67.172.10    veriqta.com
```

When to Use

- Check how your system resolves a hostname.
- Verify if /etc/hosts is overriding DNS.
- Compare results with dig.
- Troubleshoot name resolution issues on your server.

/etc/hosts and DNS

3 curl -I

- Request HTTP response headers.
- Shows status code, headers and server information.
- Useful for quickly checking if a web service is responding.
- Does not download the full content (fast).

Example

```
$ curl -I https://veriqta.com
HTTP/2 200
date: Tue, 14 Oct 2026 10:27:15 GMT
content-type: text/html
content-length: 1256
server: cloudflare
cache-control: max-age=300
strict-transport-security: max-age=31536000
```

When to Use

- Check if a website or API is up.
- Verify HTTP status codes (200, 301, 403, 500, etc.).
- Inspect important response headers.
- Useful for load balancers, CDNs and API endpoints.

https://veriqta.com

4 curl -v

- Inspect HTTP connection and request details.
- Shows detailed information about the connection, DNS lookup, TLS handshake and request/response.
- Useful for troubleshooting API and web service issues.

Example

```
$ curl -v https://veriqta.com
*   Trying 104.21.16.1:443...
* Connected to veriqta.com (104.21.16.1) port 443
* ALPN: h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
> GET / HTTP/2
> Host: veriqta.com
> user-agent: curl/8.5.0
> accept: */*
```

When to Use

- Investigate connection problems.
- See the full request and response.
- Verify TLS handshake details.
- Check redirects and headers.
- Useful for API errors and authentication issues.

https://veriqta.com

5 openssl s_client

- Inspect TLS connections and presented certificates.
- Shows certificate details, issuer, validity dates and supported ciphers.
- Useful for diagnosing HTTPS, TLS handshake and certificate issues.

Example

```
$ openssl s_client -connect veriqta.com:443
CONNECTED(00000003)
depth=2 C = US, O = Cloudflare, Inc.
verify return:1
depth=1 C = US, O = Cloudflare, Inc.
verify return:1
depth=0 CN = veriqta.com
verify return:1
---
SSL-Session:
    Protocol  : TLSv1.3
```

When to Use

- Check if a server's TLS port (usually 443) is open.
- View the server's certificate details.
- Verify certificate validity and expiry.
- Troubleshoot TLS handshake failures.
- Useful when curl fails or you need certificate information.

veriqta.com:443

Engineering Focus

Diagnose DNS resolution, TLS, HTTP, and application response failures. Use these commands to find where the problem is happening: name resolution, network, TLS, or the application itself.

Common Beginner Mistakes

- Assuming a website is down without checking DNS.
- Not checking the correct port (e.g. 443 for HTTPS).
- Ignoring TLS certificate errors.
- Not comparing dig and getent hosts results.
- Confusing HTTP errors with network or DNS issues.

Try This

1. Run dig veriqta.com and check the IP address.
2. Run getent hosts veriqta.com and compare.
3. Use curl -I https://veriqta.com and check the status code.
4. Use curl -v https://veriqta.com and review the connection.
5. Run openssl s_client -connect veriqta.com:443 and check the certificate.

Remote Access and Secure File Transfer

VERIQTA

Learn how engineers access and transfer files between Linux systems.

1 ssh

- Connect securely to a remote host.
- Uses SSH (Secure Shell) for encrypted communication.
- Allows you to run commands on a remote system.
- Commonly used for server administration and DevOps work.

Example

```
$ ssh ubuntu@192.168.1.10
The authenticity of host '192.168.1.10'
(192.168.1.10) can't be established.
ED25519 key fingerprint is
SHA256:3n7....8k9.
Are you sure you want to continue
connecting (yes/no)? yes
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux)
```

When to Use

- Access a remote Linux server.
- Run administrative commands.
- Troubleshoot issues on a remote system.
- Work securely over an encrypted connection.



ssh user@host

2 ssh-keygen

- Generate SSH key pairs.
- Creates a private key and a public key.
- Used for key-based authentication (more secure than passwords).
- Helps you set up secure access to remote systems.

Example

```
$ ssh-keygen -t ed25519 -C "veriqta"
Generating public/private ed25519 key pair.
Enter file in which to save the key
(/home/ubuntu/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in
/home/ubuntu/.ssh/id_ed25519
Your public key has been saved in
/home/ubuntu/.ssh/id_ed25519.pub
```

When to Use

- Create a new SSH key pair.
- Set up key-based login instead of passwords.
- Use a strong key type (e.g., ed25519).
- Useful for automation and secure access.



id_ed25519
id_ed25519.pub

3 ssh -v

- Debug SSH connection and authentication issues.
- Shows detailed logs about the connection process.
- Helps you identify where a connection is failing.
- Useful for troubleshooting permissions, keys, and network issues.

Example

```
$ ssh -v ubuntu@192.168.1.10
OpenSSH_9.6p1, OpenSSL 3.0.13  30 Jan 2024
debug1: Connecting to 192.168.1.10 [192.168.1.10]
debug1: Connection established.
debug1: Authenticating to 192.168.1.10:22 as 'ubuntu'
debug1: Offering public key:
/home/ubuntu/.ssh/id_ed25519
debug1: Authentication succeeded (publickey).
```

When to Use

- Investigate connection failures.
- Debug authentication problems.
- Check which key is being used.
- Identify network or permission issues.



ssh -v user@host

4 scp

- Copy files over SSH.
- Securely transfers files between local and remote systems.
- Simple and widely used for one-time file transfers.
- Useful for transferring configuration files, logs, or backups.

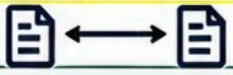
Example

```
# Copy a local file to a remote host
$ scp app.tar.gz ubuntu@192.168.1.10:/home/ubuntu/

# Copy a file from a remote host to local
$ scp ubuntu@192.168.1.10:/var/log/app.log ./
app.log        100%   2048KB    5.2MB/s    00:00
```

When to Use

- Transfer files securely between systems.
- Copy files to or from a server quickly.
- Useful for small to medium file transfers.
- Uses the same SSH authentication and encryption.



5 rsync -av

- Synchronize files while preserving selected attributes.
- Copies only changed files (more efficient than scp).
- Preserves file permissions, ownership, timestamps, and symbolic links.
- Useful for backups and keeping directories in sync.

Example

```
# Sync a local directory to a remote host
$ rsync -av /home/ubuntu/project/ \
    ubuntu@192.168.1.10:/home/ubuntu/project/
sending incremental file list
./
app/
app/config.yaml
app/main.py
sent 123,456 bytes  received 789 bytes
total size is 234,567  speedup is 1.88
```

When to Use

- Synchronize directories between systems.
- Copy only changed files.
- Preserve permissions, ownership, timestamps, and links.
- Useful for backups, deployments, and large file transfers.



rsync -av source/ dest/

Engineering Focus

Protect private keys and troubleshoot failed authentication safely. Use secure file transfer methods and verify connectivity, permissions, and network access when access or transfers fail.

Common Beginner Mistakes

- Storing private keys in insecure locations.
- Using weak permissions on private keys.
- Forgetting to add the public key to the remote server.
- Troubleshooting SSH without using `-v`.
- Using scp for large transfers instead of rsync.

Try This

1. Generate a key pair with ssh-keygen.
2. Copy your public key to a server.
3. Connect to a remote host using ssh.
4. Transfer a file using scp.
5. Synchronize a directory using rsync -av.

Git Repository Foundations

VERIQTA

Learn how DevOps engineers inspect and track changes to application and infrastructure code.

1 git clone

- Download a repository.
- Creates a local copy of a remote repository.
- Copies all files, branches, and history.
- Commonly used to get started with a project.

Example

```
$ git clone https://github.com/veriqta/demo.git
Cloning into 'demo'...
remote: Enumerating objects: 125, done.
remote: Counting objects: 100% (125/125), done.
remote: Compressing objects: 100% (80/80), done.
remote: Total 125 (delta 45), reused 125 (delta 45)
Receiving objects: 100% (125/125), 1.23 MiB | 2.34
MiB/s, done.
```

When to Use

- Get a copy of a project from GitHub or another remote repository.
- Start working on existing code.
- Useful for application and infrastructure projects.

git clone <repo-url>

2 git status

- Inspect working-tree changes.
- Shows the current branch and the state of files.
- Identifies modified, staged, and untracked files.
- Useful for understanding what will be committed.

Example

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in
  working directory)
        modified:   app.py
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        config.yaml
no changes added to commit (use "git add" and/or "git commit -a")
```

When to Use

- Check what has changed.
- Verify which files are modified or untracked.
- Confirm if files are staged before committing.
- Useful before running git add or git commit.

git status

3 git diff

- Review unstaged modifications.
- Shows the exact changes made in files.
- Helps you understand what has changed line by line.
- Useful for reviewing code before staging or committing.

Example

```
$ git diff app.py
diff --git a/app.py b/app.py
index 3a2b1c4..7d9e8f1 100644
--- a/app.py
+++ b/app.py
@@ -1,3 +1,4 @@
 print("Hello World")
+print("Hello DevOps")
```

When to Use

- See the changes you made before staging them.
- Review code differences.
- Check for accidental changes.
- Useful for catching mistakes early.

git diff <file>

4 git add

- Stage selected changes.
- Adds file changes to the staging area.
- Prepares files to be included in the next commit.
- You can add specific files or all changes.

Example

```
# Add a specific file
$ git add app.py
# Add multiple files
$ git add app.py config.yaml
# Add all changes
$ git add .
```

When to Use

- Stage files you want to commit.
- Add specific files or all changes.
- Useful after reviewing changes with git diff.
- Prepares a clean and intentional commit.

git add <file>
git add .

5 git commit

- Record a snapshot of staged changes.
- Saves your changes with a meaningful message.
- Creates a commit in the local repository history.
- Useful for tracking progress and making changes easy to review.

Example

```
$ git commit -m "Add configuration file"
[main 4f9e2d1] Add configuration file
 1 file changed, 12 insertions(+)
 create mode 100644 config.yaml
```

When to Use

- Save your staged changes.
- Use a clear and meaningful commit message.
- Keep a history of changes.
- Useful for collaboration and code reviews.

git commit -m "your message"

Engineering Focus

- Review changes and avoid committing credentials or unintended files.
- Use meaningful commit messages.
- Check .gitignore to exclude sensitive files.
- Keep your repository clean and organized.

Common Beginner Mistakes

- Committing credentials, API keys, or secrets.
- Adding unnneessary files (e.g. logs, binaries).
- Using unclear commit messages.
- Not checking changes with git status or git diff.
- Forgetting to use .gitignore.

Try This

1. Clone a repository from GitHub.
2. Make a small change to a file.
3. Check the changes with git status and git diff.
4. Stage the file using git add.
5. Commit the change with a clear message.

Git Collaboration and Change Recovery

VERIQTA

Learn how teams collaborate safely using branches and remote repositories.

1 git switch -c

- Create and switch to a new branch.
- Helps you work on changes without affecting the main branch.
- Keeps your work isolated.
- Useful for features, fixes, and experiments.

Example

```
$ git switch -c feature/login
Switched to a new branch
'feature/login'
$ git branch
  main
* feature/login
```

When to Use

- Start working on a new feature or fix.
- Keep changes separate from main.
- Use a clear and descriptive branch name.
- Common in team collaboration.



feature/login

2 git fetch

- Retrieve remote changes without merging.
- Downloads updates from the remote repository.
- Lets you see what others have pushed.
- Useful for checking changes before merging.

Example

```
$ git fetch origin
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (25/25), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 18 (delta 8), reused 0 (delta 0)
From github.com:veriqta/demo
   a1b2c3d..e4f5g6h  main       -> origin/main
   9f8e7d6..c3b2a1   feature/api -> origin/feature/api
```

When to Use

- Get the latest changes from the remote repository.
- See what your teammates have pushed.
- Check for updates before merging or pulling.
- Useful in shared repositories.



git fetch origin

3 git pull --ff-only

- Update a branch only when fast-forwarding is possible.
- Downloads and merges changes from the remote.
- Does not create a merge commit.
- Helps keep a clean and linear history.

Example

```
$ git pull --ff-only origin main
From github.com:veriqta/demo
 * branch            main     -> FETCH_HEAD
Updating a1b2c3d..e4f5g6h
Fast-forward
 app.py          |  3 +++
 README.md       |  2 ++
 2 files changed, 5 insertions(+)
```

When to Use

- Keep your local branch up to date.
- Allow the pull only if it can fast-forward.
- Avoid unwanted merge commits.
- Useful for clean and simple histories.



git pull --ff-only

4 git log --oneline

- Inspect concise commit history.
- Shows each commit on a single line.
- Displays commit hash and message.
- Useful for quickly reviewing recent changes.

Example

```
$ git log --oneline
e4f5g6h  Add login endpoint
c3b2a1d  Update configuration
9f8e7d6  Fix error handling
a1b2c3d  Initial commit
```

When to Use

- View recent commit history quickly.
- Understand what changes were made.
- Identify specific commits.
- Useful during reviews, troubleshooting, and rollbacks.



git log --oneline

5 git revert

- Create a new commit that reverses an earlier commit.
- Safely undoes changes without rewriting history.
- Commonly used in shared repositories.
- Helps maintain a clear and auditable history.

Example

```
$ git revert e4f5g6h
[main 7a8b9c0] Revert "Add login endpoint"
 1 file changed, 3 deletions(-)
```

When to Use

- Undo a commit safely.
- Use when changes have already been shared with others.
- Avoid rewriting history in collaborative environments.
- Useful for fixing issues in production or shared branches.



git revert <commit-id>

Engineering Focus

- Understand why reverting shared changes is often safer than rewriting history.
- Use branches to collaborate safely.
- Keep a clean, understandable commit history.

Common Beginner Mistakes

- Committing sensitive information (keys, passwords).
- Working directly on main instead of a branch.
- Using force push and rewriting shared history.
- Skipping git fetch before pull.
- Not understanding the difference between revert and reset.

Try This

1. Create and switch to a new branch using `git switch -c`.
2. Fetch remote changes with `git fetch`.
3. Update your branch using `git pull --ff-only`.
4. View recent commits with `git log --oneline`.
5. Revert a commit using `git revert`.

CI/CD and Build Verification

VERIQTA

Learn how engineers execute and inspect build and automation workflows.

1 make

- Run project-defined build or automation targets.
- Executes tasks defined in a Makefile.
- Commonly used to build, test, package, or run applications.
- Helps standardize and simplify repeated tasks.

Example

```
$ make
Building project...
Compiling source files...
Running tests...
Build completed successfully.

$ make test
Running tests...
All tests passed.
```

When to Use

- Build or run tasks defined in a Makefile.
- Run tests, linting, or packaging tasks.
- Automate common development tasks.
- Useful for many languages and tools.

Makefile

2 npm ci

- Install dependencies from a Node.js lockfile (package-lock.json).
- Creates a clean and reproducible installation.
- Faster and more reliable than npm install in CI environments.
- Ensures the exact dependency versions are used.

Example

```
$ npm ci
added 325 packages in 12s

found 0 vulnerabilities
```

When to Use

- Set up a clean Node.js project environment.
- Install exact versions from package-lock.json.
- Use in CI/CD pipelines for consistent builds.
- Avoids unexpected version changes.

npm package-lock.json

3 npm test

- Run project-defined tests.
- Executes the test script defined in package.json.
- Validates that the application works as expected.
- Commonly runs unit, integration, or end-to-end tests.

Example

```
$ npm test
> veriqta-app@1.0.0 test
> jest

 PASS  tests/app.test.js
  ✓ should return success (12 ms)
  ✓ should handle invalid input (8 ms)
Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Time:        1.234 s
```

When to Use

- Run the tests defined in your project.
- Verify that code changes do not break functionality.
- Use in local development and CI/CD pipelines.
- Helps catch issues early.

package.json npm

4 gh run list

- Inspect GitHub Actions workflow runs.
- Shows recent workflow runs, their status, and basic details.
- Helps you check if a build or deployment succeeded or failed.
- Useful for monitoring CI/CD activity from the command line.

Example

```
$ gh run list
STATUS       TITLE          WORKFLOW     BRANCH    EVENT
✓ Success    Build and Test ci.yml       main      push
✗ Failure    Lint           ci.yml       main      push
✓ Success    Deploy         deploy.yml   main      workflow_dispatch
! In Progress Test          ci.yml       feature   pull_request
```

When to Use

- View recent workflow runs in a repository.
- Check the status of builds, tests, and deployments.
- Useful for troubleshooting CI/CD issues.
- Quickly see which workflows ran and their results.

GitHub Actions

5 gh run view --log

- Read logs for a specific workflow run.
- Shows detailed logs of each job and step.
- Helps you diagnose build or test failures.
- Useful for finding the exact reason a workflow failed.

Example

```
$ gh run view 1234567890 --log
Build job       ✓ completed in 42s
  >> Set up job                    ✓
  >> Install dependencies          ✓
  >> Run tests                     ✓
  >> Upload artifacts              ✓
  >> Complete job                  ✓
Test job        ✗ failed in 18s
  >> Run unit tests
     Error: Expected 2 but received 1
  >> Upload test results
```

When to Use

- Read detailed logs for a workflow run.
- Find the exact error that caused a failure.
- Understand what happened in each step.
- Useful when debugging CI/CD issues.

gh run view --log

Engineering Focus

Trace a change from source code to automated build and test evidence. Review changes and confirm that builds, tests, and deployments run as expected. Understand why reverting shared changes is often safer than rewriting history.

Common Beginner Mistakes

- Using **npm install** instead of **npm ci** in CI.
- Skipping tests before deploying.
- Not checking workflow logs when a build fails.
- Assuming a green build means everything is working.
- Ignoring dependency and environment issues.

Try This

1. Run **make** and check the output.
2. Install dependencies with **npm ci**.
3. Run project tests with **npm test**.
4. List recent GitHub Actions runs with **gh run list**.
5. View detailed logs for a run with **gh run view --log**.

Docker Images and Container Lifecycle

VERIQTA

Learn the distinction between building an image and running a container.

1 docker build

- Build an image from a Dockerfile.
- Reads instructions from a Dockerfile.
- Creates a new image with a specified tag.
- Used to package your application and its dependencies into an image.

Example

```
$ docker build -t veriqta-demo:1.0 .
[+] Building 10.5s (8/8) FINISHED
 => [internal] load build definition   0.0s
 => [internal] load .dockerignore      0.0s
 => [internal] load metadata for
    docker.io/library/node:20          1.2s
 => [4/4] RUN npm install              6.3s
 => exporting to image                 0.8s
 => naming to docker.io/veriqta-demo:1.0
```

When to Use

- Create an image from your application code.
- Package applications and dependencies.
- Use a clear and meaningful tag.
- Rebuild when dependencies or code change.

Dockerfile

2 docker images

- List locally available images.
- Shows repository, tag, image ID, size and when it was created.
- Helps you confirm that your image was built successfully.
- Useful for managing and cleaning up old images.

Example

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	SIZE
veriqta-demo	1.0	a1b2c3d4e5f6	485MB
node	20	3f4e5d6c7b8a	1.1GB
nginx	latest	9c8b7a6d5e4f	192MB

When to Use

- Verify that an image exists locally.
- Check the correct tag and size.
- Identify images before running a container.
- Useful for cleanup and storage management.

docker images

3 docker run

- Create and start a container from an image.
- Runs a process inside a new container.
- Can map ports, set environment variables, and mount volumes.
- Used to run applications, databases, or tools in isolated environments.

Example

```
$ docker run -d \
  --name veriqta-web \
  -p 8080:3000 \
  veriqta-demo:1.0
f3e2d1c4b5a6978c0d1e2f3a4b5c6d7e8f9a0b1c
```

When to Use

- Start a container from an image.
- Map ports to access the application.
- Set environment variables if needed.
- Use a meaningful container name.

docker run <options> <image>

4 docker ps -a

- Inspect running and stopped containers.
- Shows container ID, name, image, status and ports.
- Helps you see the lifecycle state of each container.
- Useful for troubleshooting and managing containers.

Example

```
$ docker ps -a
```

CONTAINER ID	NAME	IMAGE	STATUS
f3e2d1c4b5a6	veriqta-web	veriqta-demo:1.0	Up 2 minutes
a1b2c3d4e5f6	db	postgres:16	Exited (0) 1 hour ago
9c8b7a6d5e4f	cache	redis:7	Exited (0) 3 hours ago

When to Use

- See both running and stopped containers.
- Check the status before starting, stopping, or removing.
- Understand why a container is not running.
- Useful for audits and troubleshooting.

docker ps -a

5 docker stop

- Request graceful container shutdown.
- Sends a SIGTERM signal to the main process in the container.
- Allows the application to stop cleanly.
- Useful for stopping containers before maintenance or updates.

Example

```
$ docker stop veriqta-web
veriqta-web
$ docker ps
CONTAINER ID   NAME          STATUS
f3e2d1c4b5a6   veriqta-web   Exited (0) 5 seconds ago
```

When to Use

- Stop a container gracefully.
- Give the application time to shut down.
- Use before updates or configuration changes.
- Remember that stopped containers remain visible with docker ps -a.

docker stop <container>

Engineering Focus

- Understand image tags, container lifecycle, and why stopped containers remain visible.
- Know the difference between building an image and running a container.
- Use containers in a safe and predictable way in development and production.

Common Beginner Mistakes

- Forgetting to specify a tag when building an image.
- Not mapping ports when running a container.
- Confusing images with containers.
- Expecting docker ps to show stopped containers.
- Not understanding that stopped containers remain visible with docker ps -a.

Try This

1. Build a simple image from a Dockerfile.
2. List your images with docker images.
3. Run a container from your image.
4. Check running and stopped containers with docker ps -a.
5. Stop the container with docker stop.

Docker Troubleshooting and Runtime Inspection

VERIQTA

Learn how to diagnose container startup failures and application problems.

1 docker logs

- Read container output.
- Shows application logs from stdout and stderr.
- Helps you identify startup errors and runtime issues.
- Useful for debugging failed or unhealthy containers.

Example

```
$ docker logs veriqta-web
> Starting application...
> Connecting to database...
> Error: Cannot connect to database at db:5432
> Application exited with code 1
```

When to Use

- Check why a container failed to start.
- View real-time application output.
- Look for errors, warnings, and stack traces.
- Use -f to follow logs in real time.

`docker logs <container>`

2 docker inspect

- Inspect container configuration and state.
- Shows detailed information in JSON format.
- Helps you verify environment variables, mounts, networks, and health status.
- Useful for understanding how a container was created and is configured.

Example

```
$ docker inspect veriqta-web
{
  "Id": "f3e2d1c4b5a6",
  "Name": "/veriqta-web",
  "State": {
    "Status": "exited",
    "ExitCode": 1
  },
  "Config": {
    "Env": ["NODE_ENV=production"],
    "Image": "veriqta-demo:1.0"
}
```

When to Use

- Check container state (running, exited, etc).
- Verify environment variables and configuration.
- Confirm the image, mounts, and network settings.
- Troubleshoot unexpected container behavior.

`docker inspect <container>`

3 docker exec

- Run a command inside a running container.
- Open an interactive shell for debugging.
- Inspect files, configurations, and running processes.
- Useful for troubleshooting application issues.

Example

```
$ docker exec -it veriqta-web sh
/app # ls
node_modules   package.json   src
/app # cat .env
NODE_ENV=production
DB_HOST=db
DB_PORT=5432
/app #
```

When to Use

- Access a running container for investigation.
- Check files and configuration.
- Run troubleshooting commands.
- Verify application behavior inside the container.

`docker exec -it <container> <command>`

4 docker stats

- Monitor container resource usage.
- Shows live CPU, memory, network, and block I/O.
- Helps you identify resource bottlenecks.
- Useful for investigating performance issues.

Example

```
$ docker stats
CONTAINER ID   NAME         CPU %   MEM USAGE / LIMIT
f3e2d1c4b5a6   veriqta-web  12.34%  256MiB / 1GiB
a1b2c3d4e5f6   db           2.10%   128MiB / 1GiB
9c8b7a6d5e4f   redis        0.50%   32MiB / 512MiB
```

When to Use

- Monitor CPU and memory usage in real time.
- Identify containers using too many resources.
- Investigate performance issues.
- Compare resource usage between containers.

`docker stats`

5 docker port

- Inspect published container ports.
- Shows how container ports map to host ports.
- Helps you verify port mapping when services are not accessible.
- Useful for troubleshooting connectivity issues.

Example

```
$ docker port veriqta-web
3000/tcp -> 0.0.0.0:0880
3000/tcp -> [::]:8080
$ docker port db
5432/tcp -> 0.0.0.0:5432
5432/tcp -> [::]:5432
```

When to Use

- Check which host ports are mapped to container ports.
- Verify that the application is exposed correctly.
- Troubleshoot connectivity and port conflicts.
- Confirm port mapping for troubleshooting and access.

`docker port <container>`

Engineering Focus

- Investigate an exited container, missing configuration, or incorrect port mapping.
- Use logs, inspect, and exec to find the root cause.
- Understand container state, resource usage, and port mappings.
- Develop a systematic approach to container troubleshooting.

Common Beginner Mistakes

- Not checking logs when a container exits.
- Not inspecting container configuration.
- Running exec on a stopped container.
- Ignoring resource usage and performance issues.
- Misunderstanding port mapping.
- Assuming the application is running when the container has exited.

Try This

1. View logs of a failed container with `docker logs`.
2. Inspect a container's configuration with `docker inspect`.
3. Open a shell inside a running container with `docker exec`.
4. Monitor live resource usage with `docker stats`.
5. Check port mappings with `docker port`.

Docker Compose, Networking, and Persistence

VERIQTA

Learn how multiple containers operate as an application stack.

1 docker compose config

- Validate and render a Compose configuration.
- Checks syntax and structure.
- Shows the final configuration after variable substitution.
- Helps you detect errors before starting services.
- Useful for understanding how your stack will be created.

Example

```
$ docker compose config
services:
  web:
    image: nginx:latest
    ports:
      - "8080:80"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: app
```

When to Use

- Validate your docker-compose.yml file.
- Check for syntax errors and missing values.
- See the final configuration after environment variables are applied.
- Useful before running services.

docker compose config

2 docker compose up -d

- Start services in the background.
- Creates networks, volumes, and containers defined in the Compose file.
- Downloads images if needed.
- Uses the -d flag to run in detached mode.
- Useful for running multi-container applications.

Example

```
$ docker compose up -d
[+] Running 3/3
 ✓ Network app_default      Created
 ✓ Volume "app_db_data"     Crarted
 ✓ Container app-db-1       Started
 ✓ Container app-web-1      Started
```

When to Use

- Start all services defined in docker-compose.yml.
- Run in the background (detached mode).
- Sets up networks and volumes automatically.
- Use when bringing up your application stack.

docker compose up -d

3 docker compose ps

- Inspect Compose-managed services.
- Shows container name, image, status, ports, and more.
- Helps you check if all services are running correctly.
- Useful for quick troubleshooting.

Example

```
$ docker compose ps
NAME      IMAGE          COMMAND              STATE   PORTS
app-web-1 nginx:latest   "nginx -g ..."       Up      0.0.0.0:8888->80/tcp
app-db-1  postgres:16    "docker-entrypoint..." Up    5432/tcp
```

When to Use

- Check the status of all services.
- Verify that containers are running.
- See exposed ports and container names.
- Useful for troubleshooting and monitoring.

docker compose ps

4 docker network inspect

- Examine Docker network configuration.
- Shows network details, connected containers, IP addresses, and driver information.
- Helps you understand how containers communicate.
- Useful for troubleshooting connectivity issues.

Example

```
$ docker network inspect app_default
[
  {
    "Name": "app_default",
    "Driver": "bridge",
    "Containers": {
      "app-web-1": {"IPv4Address": "172.20.0.3"},
      "app-db-1": {"IPv4Address": "172.20.0.2"}
    }
]
```

When to Use

- Check network details for your Compose project.
- See which containers are connected.
- Verify IP addresses and network driver.
- Troubleshoot container to container communication issues.

docker network inspect <network>

5 docker volume inspect

- Examine Docker volume configuration.
- Shows volume name, mount point, driver, and usage details.
- Helps you confirm persistent storage configuration.
- Useful for troubleshooting data and permissions issues.

Example

```
$ docker volume inspect app_db_data
[
  {
    "Name": "app_db_data",
    "Driver": "local",
    "Mountpoint": "/var/lib/docker/volumes/
      app_db_data/_data",
    "Labels": {}
  }
]
```

When to Use

- Check volume details for your Compose project.
- Verify the mount point and driver.
- Confirm that data is stored persistently.
- Troubleshoot issues with missing or incorrect data.

docker volume inspect <volume>

Engineering Focus

- Understand container-to-container communication and persistent storage.
- Learn how Compose creates networks, volumes, and services.
- Investigate connecticity and data issues using evidence.
- Build reliable, reproducible application stacks.

Common Beginner Mistakes

- Incorrect indentation in docker-compose.yml.
- Using the wrong image name or tag.
- Forgetting to expose or map ports.
- Assuming containers can communicate without a network.
- Losing data by not using persistent volumes.

Try This

1. Run docker compose config to validate your file.
2. Start the stack with `docker compose up -d`.
3. Check services with `docker compose ps`.
4. Inspect the network with `docker network inspect`.
5. Inspect the volume with `docker volume inspect`.

Kubernetes Cluster and Workload Visibility

VERIQTA

Learn to identify Kubernetes resources and inspect application health.

1 kubectl config current-context

- Identify the active Kubernetes context.
- Shows which cluster you are currently connected to.
- Helps you avoid making changes in the wrong cluster.
- Useful when you work with multiple clusters (dev, staging, production).

Example

```
$ kubectl config current-context
production-cluster
$ kubectl config get-contexts
CURRENT   NAME                 CLUSTER
*         production-cluster   prod
          staging-cluster      staging
          dev-cluster          dev
```

When to Use

- Verify which cluster you are connected to.
- Confirm the context before running any changes.
- Useful in environments with multiple clusters.
- Helps prevent accidental changes in the wrong environment.

kubectl config current-context

2 kubectl get nodes

- Inspect cluster nodes.
- Shows the list of worker and control plane nodes.
- Displays node status, roles, version, and other details.
- Helps you verify cluster health and resource availablity.
- Useful for troubleshooting scheduling and resource issues.

Example

```
$ kubectl get nodes
NAME          STATUS  ROLES          AGE  VERSION
ip-10-0-1-10  Ready   control-plane  15d  v1.29.2
ip-10-0-2-11  Ready   <none>         15d  v1.29.2
ip-10-0-3-12  Ready   <none>         15d  v1.29.2
```

When to Use

- Check if nodes are ready and healthy.
- Verify the number of nodes in the cluster.
- Confirm control plane and worker node roles.
- Useful when Pods are not scheduling or cluster resources are limited.

kubectl get nodes

3 kubectl get pods -A

- List Pods across all namespaces.
- Shows the status, namespace, name, and other details for all Pods in the cluster.
- Helps you quickly see the health of all workloads.
- Useful for identifying failing, crash loop, or pending Pods.

Example

```
$ kubectl get pods -A
NAMESPACE    NAME                       READY  STATUS            RESTARTS  AGE
kube-system  coredns-6f6b679f8-abc12    1/1    Running           0         15d
kube-system  kube-proxy-xyz89           1/1    Running           0         15d
default      web-5d8f9c7b6-ptk2m        1/1    Running           0         2d
default      api-7c9d8f6b5-qw3er        0/1    CrashLoopBackOff  5         1h
```

When to Use

- Get an overview of all Pods in the cluster.
- Find failing or restarting Pods.
- Verify if your application Pods are running.
- Useful during troubleshooting and health checks.

kubectl get pods -A

4 kubectl get deployments -A

- Inspect application Deployments.
- Shows deployments across all namespaces.
- Displays desired and available replicas, image, and age.
- Helps you verify if applications are running as expected.
- Useful for checking rollout status and scaling issues.

Example

```
$ kubectl get deployments -A
NAMESPACE    NAME     READY  UP-TO-DATE  AVAILABLE  AGE
default      web      3/3    3           3          2d
default      api      2/2    2           2          2d
kube-system  coredns  2/2    2           2          15d
```

When to Use

- Check if your application deployments are running.
- Verify the number of replicas.
- Identify deployments with unavailable replicas.
- Useful for troubleshooting rollout and update issues.

kubectl get deployments -A

5 kubectl describe pod

- Investigate Pod conditions and events.
- Shows detailed information about a specific Pod.
- Includes status, IP, containers, volumes, and recent events.
- Helps you identify the reason for failures such as ImagePullBackOff, CrashLoopBackOff, or pending state.

Example

```
$ kubectl describe pod web-5d8f9c7b6-ptk2m
Name:         web-5d8f9c7b6-ptk2m
Namespece:    default
Status:       Running
IP:           10.244.1.23
Containers:
  web:
    Image:    nginx:1.25
    State:    Running
Events:
  Type    Reason   Age   From     Message
  Normal  Pulled   2m    kubelet  Succesfully pulled image
```

When to Use

- Get detailed information about a Pod.
- Check container status, configuration, and events.
- Find the exact reason for Pod failures.
- Useful for debugging startup, crash, or scheduling issues.

kubectl describe pod <pod-name>

Engineering Focus

- Confirm the correct cluster before inspecting or changing workloads.
- Understand the relationship between nodes, namespaces, deployments, and pods.
- Use these commands to quickly identify and troubleshoot application issues.

Common Beginner Mistakes

- Running commands on the wrong cluster.
- Forgetting to use -A to see resources in all namespaces.
- Misinterpeting Pod status or events.
- Not checking recent events when a Pod fails.
- Making changes without verifying the current context.

Try This

1. Check your current context.
2. List all nodes in your cluster.
3. List all pods across namespaces.
4. Inspect deployments and verify replicas.
5. Describe a failing pod and review the events.

Kubernetes Application Troubleshooting

VERIQTA

Learn how to investigate unhealthy Pods and application connectivity.

1 kubectl logs

- Read container logs.
- Shows stdout and stderr from a Pod or container.
- Helps you identify application errors, startup failures, and runtime issues.
- Useful for troubleshooting CrashLoopBackOff and unhealthy applications.

Example

```
$ kubectl logs web-5d8f9c7b6-ptk2m
> Starting application...
> Connecting to database...
> Error: Could not connect to database at db:5432
> Application exited with code 1
```

When to Use

- Check why a container is crashing or not starting.
- View application logs in real time.
- Look for errors, warnings, and stack traces.
- Useful for investigating CrashLoopBackOff and ImagePullBackOff issues.

kubectl logs <pod-name>

2 kubectl exec

- Execute diagnostic commands in a container.
- Open a shell to investigate the running environment.
- Check files, configurations, and network connectivity.
- Useful for debugging application issues from inside the container.

Example

```
$ kubectl exec -it web-5d8f9c7b6-ptk2m -- sh
/app # ls
app.js   node_modules   package.json
/app # cat .env
DB_HOST=db
DB_PORT=5432
DB_USER=app
/app #
```

When to Use

- Inspect a running container.
- Check application files and configuration.
- Test network connectivity from inside the Pod.
- Run troubleshooting commands (curl, ping, etc).
- Useful for verifying environment variables and dependencies.

kubectl exec -it <pod> -- <command>

3 kubectl get events

- Inspect Kubernetes events.
- Shows recent events for Pods, Deployments, and other resources.
- Helps you understand why a Pod is failing, restarting, or not starting.
- Useful for identifying scheduling errors, image pull failures, and readiness check issues.

Example

```
$ kubectl get events --sort-by=.lastTimestamp
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m	Normal	Scheduled	pod/web	Successfully assigned
2m	Normal	Pulled	pod/web	Container image pulled
1m	Warning	Failed	pod/web	Readiness probe failed
1m	Warning	BackOff	pod/web	Back-off restarting failed container

When to Use

- Find recent issues in the cluster.
- See why a Pod is not starting.
- Identify scheduling, pulling, or readiness failures.
- Useful for troubleshooting CrashLoopBackOff, ImagePullBackOff, and unhealthy Pods.

kubectl get events

4 kubectl get svc

- Inspect Kubernetes Services.
- Shows service name, type, cluster IP, external IP, and ports.
- Helps you verify how your application is exposed.
- Useful for checking connectivity between services and from outside the cluster.

Example

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	15d
web	ClusterIP	10.105.18.3	<none>	80/TCP	2d
api	LoadBalancer	10.105.22.7	34.68.12.5	80:31234/TCP	2d
db	ClusterIP	10.103.9.4	<none>	5432/TCP	2d

When to Use

- Verify that a service exists.
- Check the service type and IP address.
- Confirm that the correct ports are exposed.
- Troubleshoot connectivity between services or from outside the cluster.

kubectl get svc

5 kubectl rollout status

- Check Deployment rollout progress.
- Shows the current status of the rollout.
- Helps you verify whether the new version is successfully deployed.
- Useful for detecting rollout failures and stuck deployments.

Example

```
$ kubectl rollout status deployment/web
Waiting for deployment "web" rollout to finish:
3 of 3 updated replicas are available...
deployment "web" successfully rolled out
```

When to Use

- Check if a deployment has completed rolling out.
- Verify that the new version is running.
- Identify stuck or failed rollouts.
- Useful after making changes to a deployment.

kubectl rollout status <deployment>

Engineering Focus

- Investigate CrashLoopBackOff, ImagePullBackOff, failed readiness checks, and unsuccessful rollouts.
- Use logs, events, and service details to find the root cause.
- Confirm the correct cluster before inspecting or changing workloads.

Common Beginner Mistakes

- Checking the wrong namespace.
- Looking only at logs and ignoring events.
- Forgetting to check service configuration.
- Not verifying the current cluster context.
- Making changes without understanding the impact on the running application.

Try This

1. View logs for a failing Pod.
2. Exec into a running container and inspect files.
3. Check recent events for the Pod.
4. List services and verify the exposed ports.
5. Confirm the rollout status of a deployment.

Terraform and Infrastructure as Code

VERIQTA

Learn how infrastructure definitions become controlled infrastructure changes.

1 terraform init

- Initialize the working directory and providers.
- Downloads required providers and modules.
- Sets up the .terraform directory and lock file.
- Prepares the environment before running other Terraform commands.

Example

```
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions...
- Installing hashicorp/aws v5.32.0...
- Installed hashicorp/aws v5.32.0
Terraform has been successfully
initialized!
```

When to Use

- Run this in a new project or after adding new providers or modules.
- Downloads and configures required components.
- Required before running plan or apply.

terraform init

2 terraform fmt

- Format Terraform configuration files.
- Applies a consistent style and structure.
- Improves readability and reduces errors.
- Useful when working in a team.

Example

```
$ terraform fmt
main.tf
variables.tf
outputs.tf
modules/network.tf
```

When to Use

- Keep your configuration files clean and consistent.
- Run before committing changes to version control.
- Helps maintain a standard format across the team.

terraform fmt

3 terraform validate

- Validate configuration structure.
- Checks for syntax errors and internal consistency.
- Verifies that required arguments are set.
- Useful for catching mistakes before running plan.

Example

```
$ terraform validate
✓ Success! The configuration
is valid.
```

When to Use

- Check that your configuration is correct.
- Run after making changes.
- Identifies syntax errors, missing values, and invalid references.
- Useful in CI pipelines to prevent invalid changes.

terraform validate

4 terraform plan

- Preview proposed infrastructure changes.
- Shows what will be created, updated, or destroyed.
- Helps you review changes before applying them.
- Useful for understanding the impact of your changes.

Example

```
$ terraform plan
Terraform used the selected providers
to generate the following execution plan.
+ create     aws_instance.web
~ update     aws_security_group.web
- destroy    aws_s3_bucket.old
Plan: 1 to add, 1 to change, 1 to destroy.
```

When to Use

- Preview changes before applying them.
- Check that the plan matches your expectations.
- Share the plan output for review in a team.
- Helps prevent accidental changes in production.

terraform plan

5 terraform show

- Inspect a plan file or current state representation.
- Shows detailed information about resources and changes.
- Useful for reviewing a saved plan file.
- Helps you understand the current state of your infrastructure.

Example

```
$ terraform show tfplan
# aws_instance.web will be created
+ resource "aws_instance" "web" {
    + id            = (known after apply)
    + ami           = "ami-0abcdef1234567890"
    + instance_type = "t3.micro"
    + tags = {
        + "Name" = "web-server"
    }
}
```

When to Use

- Review the details of a saved plan file.
- Inspect the current state representation.
- Understand exactly what resources will be created, updated, or destroyed.
- Useful for audits and change reviews.

terraform show

Engineering Focus

- Understand infrastructure state and how Terraform tracks real resources.
- Review plans carefully before applying changes.
- Know why changes require approval and verification.
- Use version control and peer review for infrastructure code.

Common Beginner Mistakes

- Running terraform apply without reviewing the plan.
- Not initiallzing the working directory.
- Ignoring validation errors.
- Making changes in the wrong workspace or environment.
- Not understnding the impact of resource changes.

Try This

1. Create a simple project with a provider and resource.
2. Run `terraform init`.
3. Format the files with `terraform fmt`.
4. Validate the configuration with `terraform validate`.
5. Generate and review a plan with `terraform plan`.
6. Inspect the plan file with `terraform show`.

AWS Identity, Compute, and Storage

VERIQTA

Learn practical AWS CLI commands that support basic cloud investigation.

1 aws sts get-caller-identity

- Confirm the active AWS identity.
- Shows your AWS account ID, user or role, and ARN.
- Helps you verify which account you are using.
- Useful before making any changes in AWS.

Example

```
$ aws sts get-caller-identity
{
  "UserId": "AIDAEXAMPLE1234556789",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:
   user/devops-user"
}
```

When to Use

- Verify the account you are working in.
- Confirm if you are using a user or an assumed role.
- Check your identity before investigating resources.
- Useful when working with multiple AWS accounts.

aws sts get-caller-identity

2 aws configure list

- Inspect AWS CLI configuration sources.
- Shows where credentials, region, and output format are loaded from.
- Helps you troubleshoot unexpected configuration values.
- Useful when commands are using the wrong account or region.

Example

```
$ aws configure list
```

Name	Value	Type	Location
profile	<not set>	None	None
access_key	****************ABCD	env	AWS_ACCESS_KEY_ID
secret_key	****************wXYZ	env	AWS_SECRET_ACCESS_KEY
region	us-east-1	env	AWS_DEFAULT_REGION
output	json	env	AWS_DEFAULT_OUTPUT

When to Use

- Check which credentials are being used.
- Verify the configured region and output format.
- Troubleshoot configuration issues.
- Useful when commands return unexpected results.

aws configure list

3 aws ec2 describe-instances

- Inspect EC2 instance information.
- Shows instance IDs, states, instance types, and more.
- Helps you verify if instances are running and healthy.
- Useful for basic investigation and troubleshooting.

Example

```
$ aws ec2 describe-instances
{
  "Reservations": [
    {
      "Instances": [
        {
          "InstanceId": "i-0abc123def4567890",
          "InstanceType": "t3.micro",
          "State": { "Name": "running" }
        }
      ]
    }
  ]
}
```

When to Use

- Check if your EC2 instances are running.
- View instance IDs, types, states, and key details.
- Confirm the correct region and account.
- Useful when investigating application or connectivity issues.

aws ec2 describe-instances

4 aws s3 ls

- List accessible S3 buckets or objects.
- Shows buckets in your account or the contents of a bucket.
- Helps you verify access and permissions.
- Useful for checking if files are present and accessible.

Example

```
$ aws s3 ls
2026-10-01 12:00:00  my-app-backup
2026-09-28 08:30:00  veriqta-logs
2026-09-20 16:45:00  veriqta-assets

$ aws s3 ls s3://veriqta-assets
2026-10-10 10:15:00     1024  logo.png
2026-10-10 10:20:00     2048  index.html
```

When to Use

- List all accessible S3 buckets.
- Check the contents of a specific bucket.
- Verify that you have the required permissions.
- Useful when troubleshooting missing files or access issues.

aws s3 ls

5 aws cloudwatch describe-alarms

- Inspect CloudWatch alarm definitions and states.
- Shows alarm name, state, metric, and other details.
- Helps you check if alarms are in ALARM, OK, or INSUFFICIENT_DATA.
- Useful for investigating application or infrastructure issues.

Example

```
$ aws cloudwatch describe-alarms
{
  "MetricAlarms": [
    {
      "AlarmName": "HighCPU",
      "StateValue": "ALARM",
      "MetricName": "CPUUtilization",
      "Namespace": "AWS/EC2"
    }
  ]
}
```

When to Use

- Check the state of your alarms.
- Verify alarm names, metrics, and thresholds.
- Identify alarms that are in ALARM state.
- Useful when investigating performance or availability issues.

aws cloudwatch describe-alarms

Engineering Focus

- Verify the AWS account, identity, region, and permissions before investigating infrastructure..
- Understand how credentials and regions affect command results.
- Always confirm the environment before making changes.

Common Beginner Mistakes

- Using the wrong AWS account or region.
- Not configuring credentials correctly.
- Assuming resources exist in the current region.
- Ignoring permission errors.
- Running commands without verifying the output.

Try This

1. Confirm your AWS identity with `aws sts get-caller-identity`.
2. Check your configuration with `aws configure list`.
3. List your EC2 instances with `aws ec2 describe-instances`.
4. List S3 buckets with `aws s3 ls`.
5. View your CloudWatch alarms with `aws cloudwatch describe-alarms`.

Production Diagnostics and Incident Investigation

VERIQTA

Bring the notebook together with five commands used to investigate real system behavior

1 dmesg

- Inspect kernel messages.
- Shows hardware, driver, and system events.
- Useful for identifying errors, warnings, and resource issues.
- May require sudo or appropriate permissions.

Example

```
$ dmesg | tail -n 10
[12345.678901] xfs (sda1): metadata I/O error
[12345.679012] sd 0:0:0:0: [sda] tag#5 FAILED
[12345.679123] sd 0:0:0:0: [sda] I/O error
[12345.679234] systemd[1]: Started Network Manager.
[12345.679345] IPv6: eth0: link up
[12345.679456] docker0: port 1(veth1234) entered
               forwarding state
```

When to Use

- Check for kernel errors and system events.
- Investigate hardware, disk, or driver issues.
- Useful when services fail to start or the system behaves unexpectedly.
- Use with sudo if required.

dmesg

2 vmstat

- Examine CPU, memory, swapping, and system activity.
- Shows a high-level view of system performance.
- Useful for identifying resource pressure.
- Helps confirm if the system is CPU-bound, memory-bound, or experiencing I/O wait.

Example

```
$ vmstat 1 5
procs -----------memory---------- ---swap-- -----io---- -system-- -------cpu-----
 r  b    swpd   free   buff  cache   si   so    bi    bo   in   cs us sy id wa
 1  0  102400 204800 123456 654321    0    0    12    34   56   78 20 10 60 10
 2  0  102400 200000 123500 650000    0    0    15    40   70   90 30 15 45 10
 1  0  102400 198000 123400 648000    0    0    10    30   50   70 25 10 55 10
```

When to Use

- Check overall system health.
- Identify CPU, memory, swap, and I/O pressure.
- Useful during performance investigation.
- Helps confirm resource bottlenecks.

vmstat

3 iostat

- Inspect CPU and storage I/O statistics.
- Shows device utilization, throughput, and latency.
- Helps you identify disk I/O bottlenecks.
- Useful for investigating slow applications and storage issues.

Example

```
$ iostat -xz 1 3
Device    r/s    w/s    rkB/s    wkB/s   await   svctm   %util
sda      5.20  12.30  1024.00  2048.00   15.30    2.40   45.60
sdb      0.10   0.20     8.00    16.00    2.10    0.30    1.20
nvme0n1 10.50  20.10  4096.00  8192.00   25.40    3.10   78.90
```

When to Use

- Investigate disk I/O performance.
- Check device utilization and I/O latency.
- Identify storage bottlenecks.
- Useful when applications are slow due to disk activity.

iostat

4 lsof -i

- Identify processes using network sockets.
- Shows which processes are listening or connected.
- Helps you find which application is using a specific port.
- Useful for investigating unexpected connections or port conflicts.

Example

```
$ lsof -i :80
COMMAND  PID  USER   FD   TYPE  DEVICE SIZE/OFF NODE NAME
nginx   1234  root   6u   IPv4  12345      0t0  TCP  *:80 (listen)
nginx   1234  root   7u   IPv6  12346      0t0  TCP  *:80 (listen)
```

When to Use

- Check which process is using a specific port.
- Verify that the expected application is listening.
- Find unexpected or conflicting processes.
- Useful during connectivity and access issues.

lsof -i

5 timeout

- Limit how long a diagnostic command can run.
- Prevents a command from running indefinitely.
- Useful when running potentially long or unresponsive commands.
- Helps you collect evidence safely during an incident.

Example

```
$ timeout 10s curl -I http://example.com
HTTP/1.1 200 OK
Date: Wed, 15 Oct 2026 12:00:00 GMT
Server: nginx
Content-Type: text/html
Content-Length: 1234
```

When to Use

- Limit the runtime of a command.
- Prevent hanging commands.
- Useful when testing endpoints or running long diagnostics.
- Helps keep investigations safe and controlled.

timeout

Engineering Focus

- Investigate a slow or unresponsive application without immediately restarting services.
- Collect and analyze evidence before making changes.
- Understand the relationship between CPU, memory, disk, and network.
- Use the right command for the right type of issue.
- Confirm findings with multiple sources of evidence.

Final Practice

- A simulated incident:
  - A Linux application is slow.
  - A CPU-intensive process is running.
  - An HTTP endpoint is unavailable.
  - Collect evidence using the commands in this notebook.
  - Identify the likely cause.
  - Propose a safe correction.
  - Verify that the application has recovered.

Common Beginner Mistakes

- Restarting services without checking evidence.
- Ignoring kernel errors and system logs.
- Not verifying the actual process using a port.
- Overlooking disk I/O and memory pressure.
- Running long commands without a timeout.
- Assuming the issue is not network-related without checking local resources.
- Not collecting enough evidence before making changes.